https://chatgpt.com/share/6a2935fb-e608-83eb-9f49-4d9ce4629271

Given an hughe SQL table of customers with multiple attributes (age, residence state, sex, interests, purchased volume, etc), I would like to create a small aggregate datacube, from which I could query the total number of custemrs with certain features by any axis, like age, spending etc. But such aggregate data cube grows exponentially by the number of attributes which could be queried/ filtered. Any tricks to reduce the size of such datacube in data warehouse or other fields?
Thought for a couple of seconds
Yes. What you're running into is the classic **data cube explosion** problem: if you pre-aggregate every possible combination of dimensions, cube size grows roughly like:

$$\prod_i (\mathrm{cardinality}_i + 1)$$

where `+1` represents the "ALL" rollup level for each attribute.

The main trick is: **do not materialize the full cube**. Materialize only the parts that are frequently queried, cheap to store, or hard to compute on demand.

1. Use partial / iceberg cubes

Instead of storing every possible aggregate, store only "large enough" cells.

Example:

```
GROUP BY age_bucket, state, sex, interest, spending_bucket
HAVING COUNT(*) >= 100
```

This is often called an **iceberg cube**.

It works well when many combinations are sparse, for example:

```
age=73, state=Wyoming, sex=F, interest=surfing, spending=high
```

may have almost no customers, so storing it is not useful.

You can keep detailed rare combinations in the base table and only pre-aggregate dense/popular regions.

2. Bucket continuous attributes

Do not cube on raw values like exact age, exact purchase volume, exact income, exact timestamp.

Instead use bins:

```
age: 18–24, 25–34, 35–44, ...
spending: low, medium, high, VIP
purchase volume: 0, 1–5, 6–20, 21+
```

This reduces cardinality massively.

Bad:

```
age = 37
spending = 483.72
```

Better:

```
age_bucket = '35-44'
spending_bucket = 'medium'
```

You can keep the raw values in the fact table and aggregate by buckets in the cube.

3. Use dimension hierarchies

Many dimensions naturally roll up:

```
city → state → country
day → month → quarter → year
age → age group
product → category → department
```

Instead of materializing all raw combinations, materialize common hierarchy levels.

For example, this is often useful:

```
GROUP BY state, age_bucket, sex
```

But this may be too detailed:

```
GROUP BY city, exact_age, sex, exact_interest,
exact_purchase_amount
```

A well-designed star schema with dimension hierarchies can reduce cube size while preserving useful drill-down paths.

4. Materialize only common cuboids

A full cube contains every subset of dimensions:

```
(age)
(state)
(sex)
(age, state)
(age, sex)
(state, sex)
(age, state, sex)
...
```

But most applications only query a small subset.

So you might precompute only:

```
(state)
(age_bucket)
(spending_bucket)
(state, age_bucket)
(state, sex)
(age_bucket, spending_bucket)
(state, age_bucket, sex)
```

and compute rarer combinations directly from the base table.

This is called **partial materialization** or **aggregate selection**.

A practical approach:

1. Log real queries.
2. Find the top 20–100 most common grouping/filtering patterns.
3. Materialize only those.
4. Let the SQL engine scan/aggregate the base table for rare queries.

5. Use approximate query processing

For “how many customers match these filters?” you may not always need an exact answer.

You can use sketches such as:

```
HyperLogLog — approximate distinct counts
Count-Min Sketch — approximate frequency counts
Bloom filters — membership/filter acceleration
Quantile sketches — approximate percentiles
```

For example, if you want distinct customer counts across many segments, storing a **HyperLogLog sketch per segment**can be much smaller than storing exact customer sets.

This is especially useful when queries are exploratory:

```
approximately how many high-spending women aged 25–34 in
California like hiking?
```

6. Store compressed bitmaps per attribute value

For customer filtering, bitmap indexes are extremely useful.

Instead of storing every aggregate combination, store one bitmap per attribute value:

```
state=CA        → bitmap of matching customer IDs
sex=F           → bitmap
age_bucket=25-34 → bitmap
interest=hiking → bitmap
spending=high   → bitmap
```

Then a query becomes bitmap operations:

```
state=CA AND sex=F AND age_bucket=25-34 AND interest=hiking
```

is computed as:

```
bitmap(state=CA)
AND bitmap(sex=F)
AND bitmap(age_bucket=25-34)
AND bitmap(interest=hiking)
```

Then count the set bits.

This avoids materializing the exponential cube. Storage grows roughly with:

```
number of distinct attribute values
```

rather than:

```
number of combinations of attribute values
```

This is one of the best tricks for your use case.

Technologies/ideas related to this:

```
bitmap indexes
Roaring bitmaps
compressed bitsets
columnar storage
OLAP engines
inverted indexes
```

Engines like Apache Pinot, Apache Druid, ClickHouse, DuckDB, Elasticsearch/OpenSearch, and many columnar warehouses exploit variants of this idea.

7. Use columnar storage and late materialization

Modern analytical databases are very good at this type of query if the table is stored column-wise.

A query like:

```
SELECT COUNT(*)
FROM customers
WHERE state = 'CA'
  AND sex = 'F'
  AND age_bucket = '25-34'
  AND spending_bucket = 'high';
```

does not necessarily need a cube. A columnar engine can scan only the relevant columns:

```
state
sex
age_bucket
spending_bucket
```

and skip the rest.

With compression, dictionaries, min/max statistics, zone maps, and partition pruning, this may be fast enough without a full cube.

8. Partition and cluster the base table

If common filters are predictable, physically organize the table.

Examples:

```
partition by country or state
cluster/sort by age_bucket, spending_bucket
cluster by customer segment
```

This helps engines skip irrelevant data.

For example, if most queries filter by `state`, partitioning or clustering by `state` can reduce scans dramatically.

But avoid over-partitioning. Partitioning by high-cardinality fields like customer ID usually hurts.

9. Separate low-cardinality and high-cardinality dimensions

Low-cardinality dimensions are cube-friendly:

```
sex: 2–5 values
state: ~50 values
age_bucket: ~10 values
spending_bucket: ~5 values
```

High-cardinality dimensions are dangerous:

```
customer_id
email
exact location
exact income
exact purchase amount
free-text interests
```

For high-cardinality dimensions, prefer:

```
bucketing
top-K values + OTHER
search index
bitmap/inverted index
```

```
on-demand aggregation
```

Do not put high-cardinality raw dimensions into a full cube.

10. Use top-K and “other” grouping

For interests, products, residence city, etc., many values may be rare.

Instead of cubing all interests:

```
hiking
skiing
chess
surfing
classical music
retro gaming
...
```

you might keep:

```
top 100 interests
OTHER
```

Or maintain separate cubes per major category.

This is common in analytics systems because long-tail categorical data causes huge sparse cubes.

11. Use cuboid lattice pruning

In OLAP theory, the full data cube is a lattice of group-bys. You can prune it.

Useful pruning rules:

```
Do not materialize cuboids with very high estimated
cardinality.
Do not materialize cuboids rarely used by queries.
Do not materialize cuboids that can be cheaply derived from
another materialized cuboid.
Prefer cuboids that answer many workloads.
Prefer cuboids with good compression ratio.
```

Example:

If you already store:

```
GROUP BY state, age_bucket, sex
```

then you can derive:

```
GROUP BY state, sex
GROUP BY age_bucket, sex
GROUP BY state
```

from that aggregate, as long as measures are additive, such as `COUNT` or `SUM`.

12. Use additive measures when possible

Counts and sums are easy to roll up.

Good measures:

```
COUNT(*)
SUM(purchased_volume)
SUM(revenue)
```

Harder measures:

```
COUNT(DISTINCT customer_id)
MEDIAN(spending)
PERCENTILE(age)
```

For hard measures, you may need sketches or raw-level recomputation.

For your example, “total number of customers” is friendly if each customer appears once. If customers appear multiple times, then distinct-count logic is harder and HyperLogLog may help.

13. Consider a semantic layer / aggregate awareness

Many BI/data warehouse systems support **aggregate awareness**: users query a logical table, while the engine decides whether to use:

```
precomputed aggregate
materialized view
base fact table
sketch table
```

This avoids exposing cube complexity to users.

Tools and patterns include:

```
materialized views
aggregate tables
query rewrite
semantic layers
OLAP cubes
rollup tables
```

Practical design for your case

I would probably use a hybrid design:

Base table

```
customers(
  customer_id,
  age,
  age_bucket,
  state,
  sex,
  interests,
  spending,
  spending_bucket,
  purchased_volume,
  ...
)
```

Stored in a columnar warehouse.

Small aggregate tables

Precompute only common summaries:

```
GROUP BY state;
GROUP BY age_bucket;
GROUP BY spending_bucket;
GROUP BY state, age_bucket;
GROUP BY state, sex;
GROUP BY age_bucket, spending_bucket;
GROUP BY state, age_bucket, sex;
```

Bitmap / inverted indexes

For interactive ad-hoc filtering:

```
state=CA
sex=F
age_bucket=25-34
interest=hiking
spending_bucket=high
```

Combine bitmaps at query time.

Optional iceberg cube

Store only combinations where:

```
count >= threshold
```

For example:

```
GROUP BY state, age_bucket, sex, interest, spending_bucket
HAVING COUNT(*) >= 50;
```

Rare combinations fall back to base-table scan or bitmap calculation.

The most important trick

For your exact question — "count customers with certain features by any axis" — the best conceptual alternative to a full cube is usually:

Use compressed bitmap indexes or inverted indexes for each attribute value, then combine them at query time.

That gives you flexible filtering without exponential cube growth.

Full cube:

```
storage ≈ combinations of attributes
```

Bitmap approach:

```
storage ≈ distinct values across attributes
```

For customer segmentation/counting, that is often the decisive difference.

Given an hughe SQL table of customers with multiple attributes (age, residence state, sex, interests, purchased volume, etc), I would like to create a small aggregate datacube, from which I could query the total number of custemrs with certain features by any axis, like age, spending etc. But such aggregate data cube grows exponentially by the number of attributes which could be queried/filtered. Any tricks to reduce the size of such datacube in data warehouse or other fields?

Thought for a couple of seconds

Yes. What you're running into is the classic **data cube explosion** problem: if you pre-aggregate every possible combination of dimensions, cube size grows roughly like:

$$\prod_i (cardinality_i + 1)$$

where `+1` represents the "ALL" rollup level for each attribute.

The main trick is: **do not materialize the full cube**. Materialize only the parts that are frequently queried, cheap to store, or hard to compute on demand.

1. Use partial / iceberg cubes

Instead of storing every possible aggregate, store only "large enough" cells.

Example:

```sql
GROUP BY age_bucket, state, sex, interest, spending_bucket
HAVING COUNT(*) >= 100
```

This is often called an **iceberg cube**.

It works well when many combinations are sparse, for example:

```
age=73, state=Wyoming, sex=F, interest=surfing, spending=high
```

may have almost no customers, so storing it is not useful.

You can keep detailed rare combinations in the base table and only pre-aggregate dense/popular regions.

2. Bucket continuous attributes

Do not cube on raw values like exact age, exact purchase volume, exact income, exact timestamp.

Instead use bins:

```
age: 18–24, 25–34, 35–44, ...
spending: low, medium, high, VIP
purchase volume: 0, 1–5, 6–20, 21+
```

This reduces cardinality massively.

Bad:

```
age = 37
spending = 483.72
```

Better:

```
age_bucket = '35–44'
spending_bucket = 'medium'
```

You can keep the raw values in the fact table and aggregate by buckets in the cube.